## Lamport's Logical Clocks

People use physical time to order events. For example, we say that an event at 8:15 AM occurs before an event at 8:16 AM. In distributed systems, physical clocks are not always precise, so we can't rely on physical time to order events. Instead, we can use logical clocks to create a partial or total ordering of events.

- Lamport proposed the following scheme to order events in a distributed system using logical clocks. The execution of a process is characterized by a sequence of events. Depending on the application, the execution of a procedure could be one event or the execution of an instruction could be one event. When processes exchange messages constitutes one event and receiving a message constitutes one event.

## A Partial Ordering

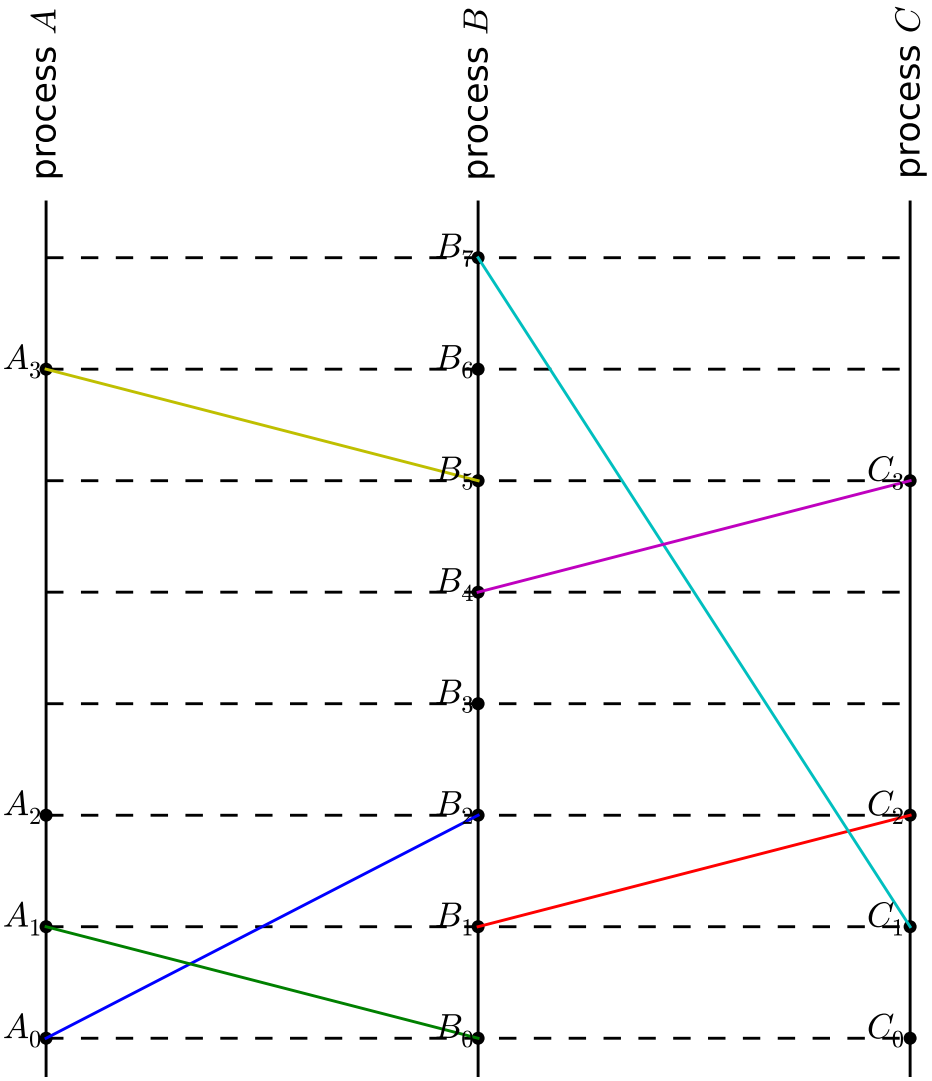
Physical time forms a natural "happened-before" irreflexive partial ordering of events. If we consider two events a and b, we can use physical time to know whether a happened before b, b happened before a

, or the two are unrelated (i.e. they happened at the same time). Since physical clocks are imprecise, we can't use physical time in a distributed system, but we'd still like an irreflexive partial ordering of events.

We'll define such an ordering, but first, let's formalize our system a bit. Our distributed system consists of a set of processes that each execute their own set of events. There are three events each process can execute:

- A local event
- Sending a message to another process
- Receiving a message from another process

The union of every process' events is the set of events we wish to order. We can visualize such a system using a space-time graph, such as the one in Figure 1 below. Each vertical line represents a process. Time flows forward as we traverse the graph upwards through the set of events represented as annotated points. If one process sends a message and another receives a message, the two are events are connected by a colored line.


process $A$
process $B$
process $C$
$B_7$
$A_3$
$B_6$
$B_5$
$C_3$
$B_4$
$B_3$
$A_2$
$B_2$
$C_2$
$A_1$
$B_1$
$C_1$
$A_0$
$B_0$
$C_0$

The system in Figure 1 has three processes: A, B, and C. Process A has four events.

A0 : Process A sends a message to process B

- A1 : Process A receives a message from process B
- A2 : Process A executes a local event
- A3 : Process A receives a message from process B

Now we can define our "happened-before" irreflexive partial ordering, denoted $\rightarrow$, as the smallest relation on events that satisfies the following three properties.

If a and b are events in the same process and a happens before b then $a \rightarrow b$. For example, $A0 \rightarrow A1$

- If a process sends a message as event a and another process receives the message as event b, then $a \rightarrow b$. For example, $A0 \rightarrow B2$
- If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$ For example, $A0 \rightarrow B2$ and $B2 \rightarrow B4$ and $B4 \rightarrow C3$, so $A0 \rightarrow C3$.

Graphically, a → b if we can trace a path forward through time from a to b. For example, we can show that A0 → C3 by tracing from A0 across the blue line to B2 up to B4 and across the purple line to C3. This interpretation also makes it clear that some events such as A2 and B3 are not related because we can't trace a path forward through time from A2 to B3 or from B3 to A2. In terms of physical time, A2 might happen before B3, but our → relation is defined independent of physical time.

Logical Clocks

Now let's introduce clocks to help us order events according to our →

ordering. Unlike physical clocks which are physical entities that assign physical times to events, our clocks are simply a conceptualization of a mathematical function that assigns numbers to events. These numbers act as timestamps that help us order events. Since our clocks logically order events, rather than physically order them, we call them logical clocks.

More formally, each process Pi has a clock Ci which is a function from events to the integers. The timestamp of an event e in Pi is Ci(e). The system clock, C, is also a function from events to the integers where C(e)=Ci(e) when e is an event in Pi. In Figure 1, timestamps are associated with horizontal dashed lines beginning at 0 and increasing by 1 forwards through time. For example, events B0 through B7 have timestamps 0 through 7

We evaluate our logical clocks with the following correctness criterion known as the Clock Condition.

- $\forall a,b.a \rightarrow b \Longrightarrow C(a)<C(b)$

  For example, $A0 \rightarrow B2$

- , so in order for a clock C to satisfy the Clock Condition, it must be that C(A0)<C(B2). Also note that it is not the case that $\forall a,b.C(a)<C(b) \Longrightarrow a \rightarrow b$. Consider A2 and B3. C(A2)<C(B3), yet A2 and B3 are not related according to our $\rightarrow$ ordering.

Given a system, we can construct a logical clock using a simple algorithm. Each process Pi maintains a mutable timestamp Ci. When an event e occurs, we let Ci(e) be Ci when e occurs. Each process Pi increments Ci after every event in Pi. Also, when a process Pi sends a message to Pj, it includes its timestamp Ci with the message. When Pj receives the message, it sets its timestamp Cj to be greater than the received Ci.

Casual ordering of messages

The purpose of causal ordering of messages is to insure that the same causal relationship for the "message send" events correspond with "message receive" events. (i.e. All the messages are processed in order that they were created.)

Birman-Schiper-Stephenson Protocol

- There are three basic principles to this algorithm:

1. All messages are time stamped by the sending process.

[Note: This time is separate from the global time talked about in the previous sections. Instead each element of the vector corresponds to the number of messages sent (including this one) to other processes.]

2. A message can not be delivered until:

- All the messages before this one have been delivered locally.
- All the other messages that have been sent out from the original processs has been accounted as delivered at the receiving process.

3. When a message is delivered, the clock is updated.

This protocol requires that the processes communicate through broadcast messages since this would ensure that only one message could be received at any one time (thus concurrently timestamped messages can be ordered).

Schiper-Eggli-Sandoz Protocol

- Instead of maintaining a vector clock based on the number of messages sent to each process, the vector clock for this protocol can increment at any rate it would like to and has no additional meaning related to the number of messages currently outstanding.

  Sending a message:

  All messages are timestamped and sent out with a list of all the timestamps of messages sent to other processes.

- Locally store the timestamp that the message was sent with.

Receiving a message:

A message cannot be delivered if there is a message mentioned in the list of timestamps that predates this one.

- Otherwise, a message can be delivered, performing the following steps:

1. Merge in the list of timestamps from the message:

- Add knowledge of messages destined for other processes to our list of processes if we didn't know about any other messages destined for one already.

- If the new list has a timestamp greater than one we already had stored, update our timestamp to match.

2. Update the local logical clock.

3. Check all the local buffered messages to see if they can now be delivered.